Spike: Task 22

Title: Collision

Author: Mohamed Shafi Uzman Fassy, 102608927

Goals / deliverables:

The goal of this spike task is to demonstrate how I use physics in 2D games to handle collision between moving entities / objects.

Besides this report, what else was created?

- Code files in see COS30031-2023-102608927\22 - Spike - Collisions\Sprite and Graphics Collision\Sprite and Graphics Task

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- SDL2 Library

Tasks undertaken:

- Project setup for SDL2 library dependencies
- Using Task 17 code to extend for collision.

Rendering Shapes & Moving Mechanisms:

I create two primary types of shapes: boxes (rectangles) and circles using classes. SDL, being a low-level library, provides direct support for rendering rectangles through its API but doesn't natively support circles. Hence, the function SDL_RenderDrawCircle has been custom-made to render a circle. It does this by iterating over a square's points and only drawing those within the circle's radius.

```
// Renders static box and circle
SDL_SetRenderDrawColor(renderer, 255, 255, 255, 255);
SDL_RenderFillRect(renderer, &staticBox.rect);
SDL_RenderDrawCircle(renderer, staticCircle.center.x, staticCircle.center.y, staticCircle.radius);
```

The movement is achieved through the SDL event loop, which listens for keyboard events. For the moving rectangle, arrow keys are used, where each key press translates the box by

10 pixels in the corresponding direction. For the moving circle, the WASD keys perform a similar function, moving the circle by 10 pixels.

```
if (event.type == SDL_KEYDOWN) {
    switch (event.key.keysym.sym) {
    case SDLK_UP:    movingBox.rect.y -= 10; break;
    case SDLK_DOWN:  movingBox.rect.y += 10; break;
    case SDLK_LEFT:  movingBox.rect.x -= 10; break;
    case SDLK_RIGHT: movingBox.rect.x += 10; break;
    case SDLK_w:     movingCircle.center.y -= 10; break;
    case SDLK_s:     movingCircle.center.y += 10; break;
    case SDLK_a:     movingCircle.center.x -= 10; break;
    case SDLK_d:     movingCircle.center.x += 10; break;
    }
}
```

Implementing Collision Manager:

```
class Box {
public:
    SDL_Rect rect;

    bool Intersects(const Box& other) const {
        return rect.x < other.rect.x + other.rect.w &&
            rect.x + rect.w > other.rect.x &&
            rect.y < other.rect.y + other.rect.h &&
            rect.y + rect.h > other.rect.y;
    }
};

class Circle {
public:
    Vector2 center;
    float radius;

    bool Intersects(const Circle& other) const {
        float dx = center.x - other.center.x;
        float dy = center.y - other.center.y;
        float distance = sqrt(dx * dx + dy * dy);
        return distance < (radius + other.radius);
    }
};
```

I did not implement a dedicated Collision Manager but instead I check its collision detection mechanism within the separate shape classes (**Box** and **Circle**) that have their own **Intersects** method and that serves as a collision manager for each shape type.

- **Box Collision**: The **Intersects** method for boxes checks if two rectangles overlap. It uses the separating axis theorem for axis-aligned bounding boxes, a common technique for this purpose.
- **Circle Collision**: The circle's **Intersects** method calculates the distance between the centers of the two circles. If this distance is less than the sum of their radii, the circles are intersecting.

Implementing Collision Testing:

```
bool boxCollided = movingBox.Intersects(staticBox);
bool circleCollided = movingCircle.Intersects(staticCircle);
```

```
// Renders moving box with collision visualization
SDL_SetRenderDrawColor(renderer, 255, 255, 255, SDL_ALPHA_OPAQUE);
SDL_RenderFillRect(renderer, &movingBox.rect);

// Renders moving circle with collision visualization
SDL_SetRenderDrawColor(renderer, boxCollided ? 255 : 0, 255, 0, SDL_ALPHA_OPAQUE);
SDL_RenderDrawCircle(renderer, movingCircle.center.x, movingCircle.center.y, movingCircle.radius);
```

In the main game loop, after processing user inputs for movement, the program tests for collisions. It checks whether the **movingBox** intersects with **staticBox** and whether **movingCircle** intersects with **staticCircle**. The results of these tests are stored in the **boxCollided** and **circleCollided** variables, respectively.

The visualization on the screen helps understand these collisions. When a collision is detected, the program changes the rendering color of the moving shapes to indicate the collision event.

Outcome:

Screen turns yellow when collision occurs when two boxes collide. While if there is not collision it turns back to green and if box collides with circle.